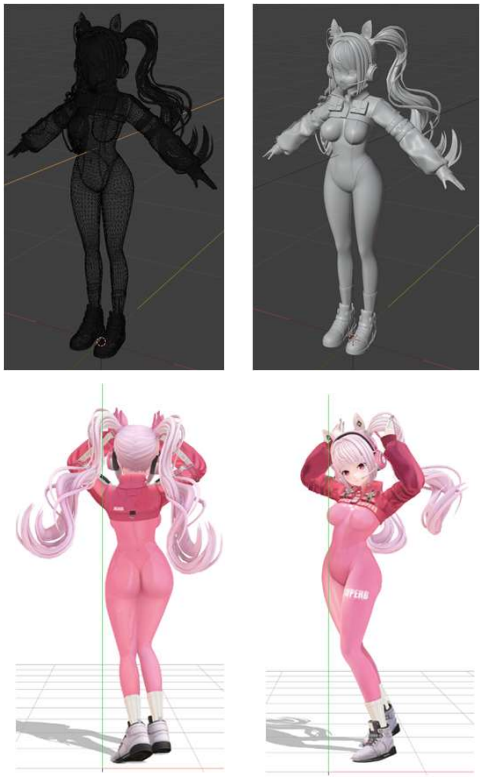
Directx3D11 프로그래밍

1. 렌더링 파이프 라인

제작자 : 이창진
이메일 : ckdwls525@gmail.com

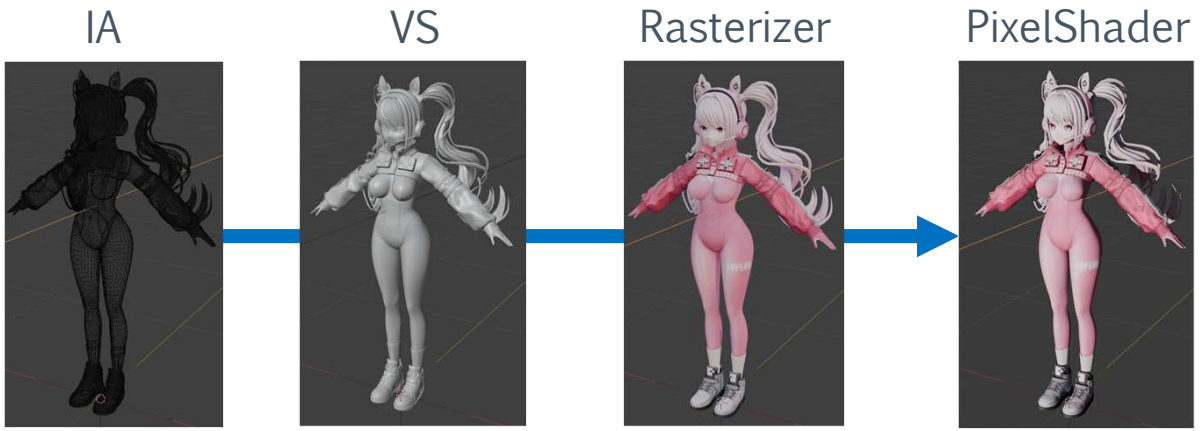
1.렌더링 파이프라인

렌더링 파이프라인에서 각 스테이지가 어떤 역할을 하는가



학습목표

```
void App::Render()
{
float color[4] = { 0.0f, 0.0f, 0.0f, 1.0f };

m_pDeviceContext-
>ClearRenderTargetView(m_pRenderTargetView, color);
// 1 ~ 3 . IA 단계 설정
// 정점을 어떻게 이어서 그릴 것인지를 선택하는 부분
m_pDeviceContext-
>IASetPrimitiveTopology(D3D11_PRIMITIVE_TOPOLOGY_T
RIANGLELIST);
// 1. 버퍼를 잡아주기
m_pDeviceContext->IASetVertexBuffers(0, 1,
&m_pVertexBuffer, &m_VertextBufferStride,
&m_VertextBufferOffset);
// 2. 입력 레이아웃을 잡아주기
m_pDeviceContext->IASetInputLayout(m_pInputLayout);
// 3. 인덱스 버퍼를 잡아주기
m_pDeviceContext->IASetIndexBuffer(m_pIndexBuffer,
DXGI_FORMAT_R16_UINT, 0);

// 4. Vertex Shader 설정
m_pDeviceContext->VSSetShader(m_pVertexShader,
nullptr, 0);

// 4. Pixel Shader 설정
m_pDeviceContext->PSSetShader(m_pPixelShader,
nullptr, 0);

// 6. 그리기
m_pDeviceContext->DrawIndexed(m_nIndices, 0, 0);

m_pSwapChain->Present(0, 0);
}
```

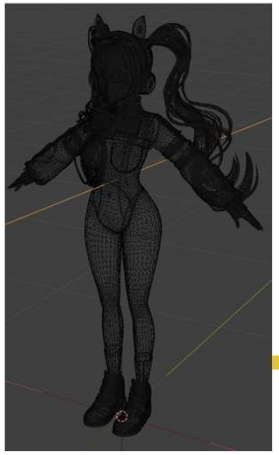


- 1. 렌더링 파이프 라인을 이해합니다
- 2. Input-Assembler를 이해합니다
- 3. VertexShader를 이해합니다
- 4. Rasterizer를 이해합니다
- 5. PixelShader를 이해합니다

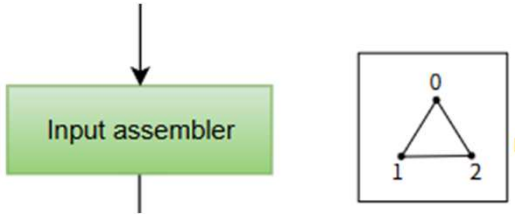
Input-Assembler Stage

- 파이프라인의 첫 번째 단계 (IA)
- 메모리(버텍스 버퍼/인덱스 버퍼)에서 정점 데이터를 읽어 **Vertex Shader로 전달할 실제 Vertex** 를 조립합니다. 튜토리얼 앱의 bool App::InitD3D() 함수 부분을 참고.
- 버텍스의 구조, 어떻게 그릴지, 버텍스 버퍼, 인덱스 버퍼를 설정합니다.
- 랜더링할때 버퍼를 조합해서 Vertex Shader에 전달할 버텍스 정보를 만듭니다.
- 버텍스는 **하나 이상의 버퍼**에서 조립 가능 (보통 하나에 Position+UV+Color 포함)

3D model



```
// 정점을 어떻게 이어서 그릴 것인지를 선택하는 부분
m_pDeviceContext->IASetPrimitiveTopology(D3D11_PRIMITIVE_TOPOLOGY_TRIANGLELIST);
m_pDeviceContext->IASetVertexBuffers(0, 1, &m_pVertexBuffer, &m_VertextBufferStride,
&m_VertextBufferOffset);
m_pDeviceContext->IASetInputLayout(m_pInputLayout);
m_pDeviceContext->IASetIndexBuffer(m_pIndexBuffer, DXGI_FORMAT_R16_UINT, 0);
```



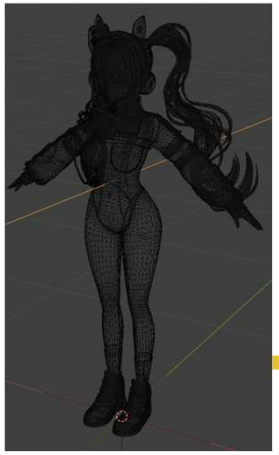
2D Space

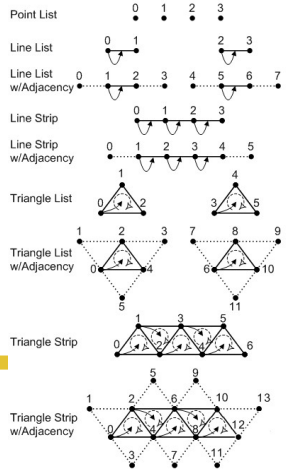
IA [PrimitiveTopology]

- **Primitive** :
  - 그래픽스 파이프라인에서 그려지는 기본 도형 단위입니다
  - 점(Point), 선(Line), 삼각형(Triangle) 등을 통칭합니다
- **주요 Primitive Topology**
  - Point List : 독립적인 점들
  - Line List : 독립적인 선분 (두 점 = 한 선)
  - Line Strip : 연결된 선분들
  - Triangle List : 독립적인 삼각형 (세 점 = 한 삼각형) → 가장 많이 사용
  - Triangle Strip : 연결된 삼각형들 (정점 재사용)

- **Winding Direction (정점 나열 방향)** :
  - 삼각형의 앞/뒤 판정 기준
  - 기본값: **CW = Front face**, CCW = Back face
- **Strip 생성과 Restart**
  - Line Strip, Triangle Strip 등은 여러 도형을 공유 정점으로 연결 가능
  - 특수 인덱스 -1 (0xFFFF / 0xFFFFFFFF) 또는 RestartStrip() 호출로 새 Strip 시작 가능
- **ID3D11DeviceContext::IASetPrimitiveTopology(topology);**
  - Input Assembler 단계에서 사용할 Primitive Topology 지정

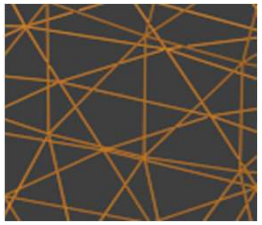
3D model





2D Space

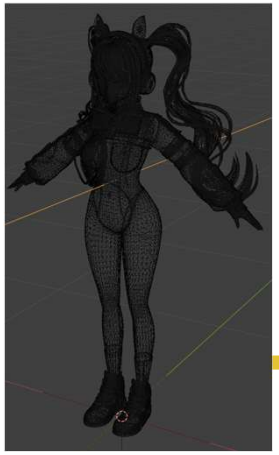
IA [Vertex Buffer]

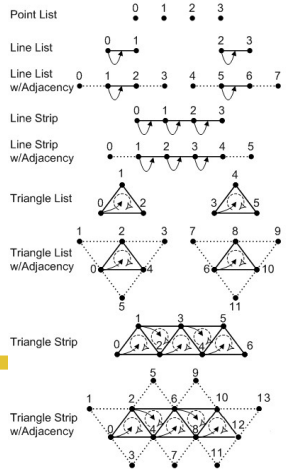


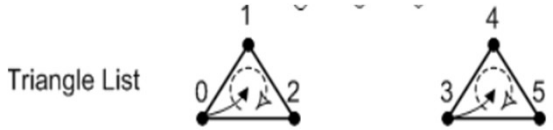
- **Vertex Buffer** :
  - 정점(Vertex) 데이터를 담는 GPU 메모리 버퍼.
  - 정점 정보: 위치(Position), 색상(Color), 법선(Normal), UV 등
  - 이 정보를 Vertex Shader에게 전달합니다
- **주의사항**
  - 나의 버퍼에 위치+UV+색상 등 모든 속성을 담을 수 있음
  - 여러 개의 버퍼에 속성을 분리해서 담을 수도 있음 (예: 위치 /UV 따로)
  - 버퍼 연결 시 슬롯(slot) 번호로 지정 → IASetVertexBuffers()

```
VertexInfo vertices[] =
{
VertexInfo(Vector3(-0.5f,  0.5f, 0.5f), Vector4(1.0f, 0.0f, 1.0f, 1.0f)),
VertexInfo(Vector3(0.5f,  0.5f, 0.5f), Vector4(0.0f, 1.0f, 0.0f, 1.0f)),
VertexInfo(Vector3(-0.5f, -0.5f, 0.5f), Vector4(1.0f, 0.2f, 1.0f, 1.0f)),
VertexInfo(Vector3(0.5f, -0.5f, 0.5f), Vector4(0.0f, 0.6f, 1.0f, 1.0f))
};

D3D11_BUFFER_DESC vbDesc = {};
vbDesc.ByteWidth = sizeof(VertexInfo) * ARRAYSIZE(vertices);
vbDesc.BindFlags = D3D11_BIND_VERTEX_BUFFER;
vbDesc.Usage = D3D11_USAGE_DEFAULT;
D3D11_SUBRESOURCE_DATA vbData = {};
vbData.pSysMem = vertices;// 배열 데이터 할당.
HR_T(m_pDevice->CreateBuffer(&vbDesc, &vbData, &m_pVertexBuffer));
m_VertextBufferStride = sizeof(VertexInfo);// 버텍스 버퍼 정보
m_VertextBufferOffset = 0;
```

3D model

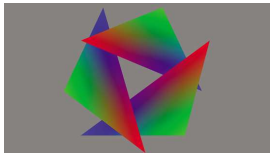


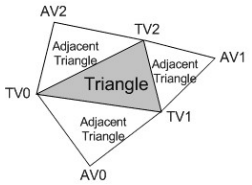


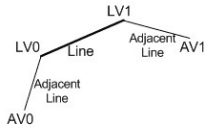


2D Space

Vertex shader Stage

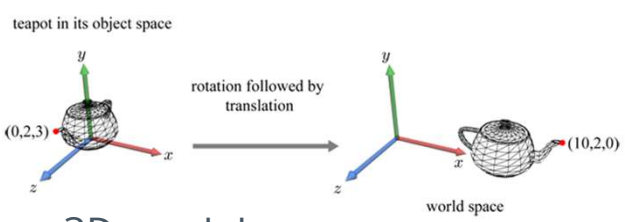


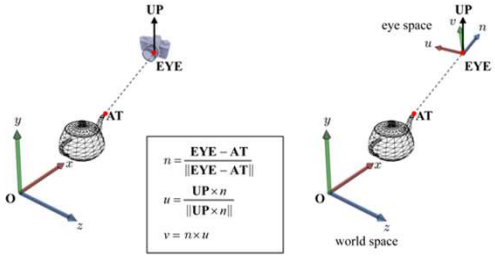




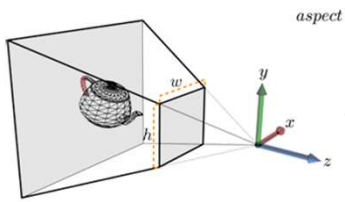
- 파이프라인에서 **IA(Input Assembler)** 가 전달한 정점을 처리하는 단계. **정점 단위(per-vertex) 연산** 수행합니다
- 아무 작업도 하지 않더라도 **패스스루(Vertex → Vertex)** 셰이더를 작성해 넣어야 합니다
- **표 변환** : 모델 공간 → 월드 공간 → 뷰 공간 → 프로젝션(클립) 공간 변환과 스키닝, 조명 같은 연산을 수행한다.

```
m_pDeviceContext->VSSetShader(m_pVertexShader, nullptr, 0);
```

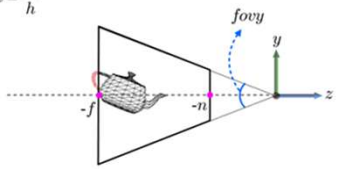




$$n = \frac{\mathbf{EYE} - \mathbf{AT}}{\|\mathbf{EYE} - \mathbf{AT}\|}$$
$$u = \frac{\mathbf{UP} \times n}{\|\mathbf{UP} \times n\|}$$
$$v = n \times u$$

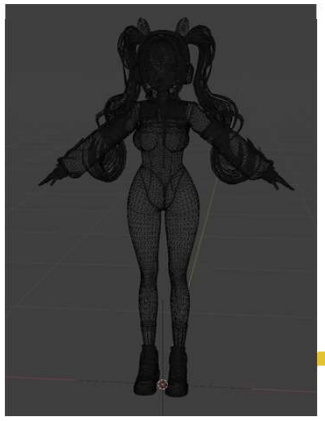


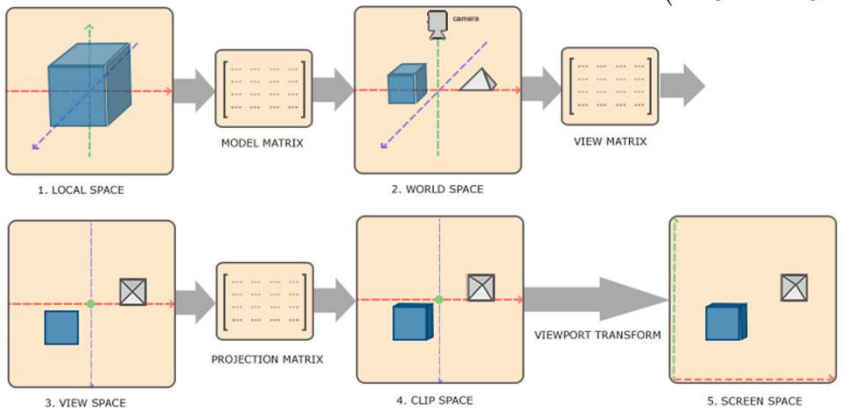
$$aspect = \frac{w}{h}$$



$$\begin{pmatrix} \frac{cot(\frac{fovy}{2})}{aspect} & 0 & 0 & 0 \\ 0 & cot(\frac{fovy}{2}) & 0 & 0 \\ 0 & 0 & -\frac{f+n}{f-n} & -\frac{2nf}{f-n} \\ 0 & 0 & -1 & 0 \end{pmatrix}$$

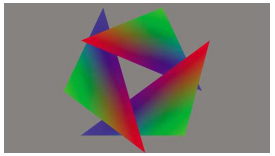
3D model





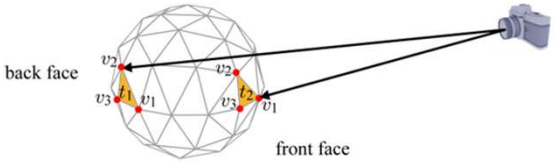
2D Space

Rasterizer Stage

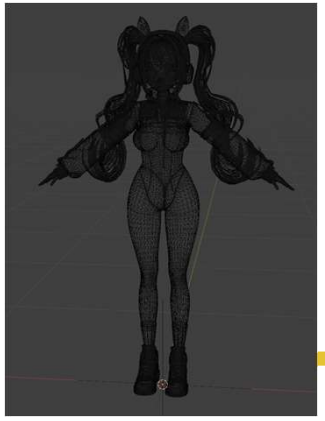


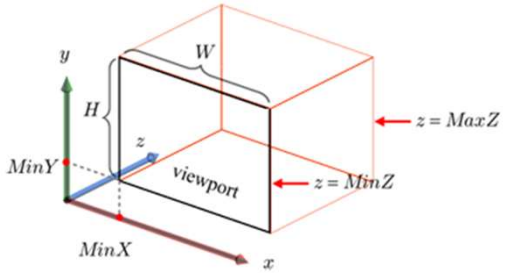
```
D3D11_VIEWPORT viewport = {};
viewport.TopLeftX = 0;
viewport.TopLeftY = 0;
viewport.Width = (float)m_ClientWidth;
viewport.Height = (float)m_ClientHeight;
viewport.MinDepth = 0.0f;
viewport.MaxDepth = 1.0f;
m_pDeviceContext->RSSetViewports(1,
&viewport);
```

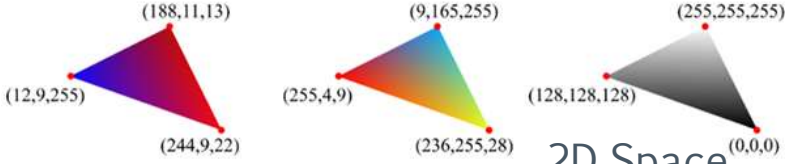
- **벡터(정점/프리미티브)를 픽셀(Fragment)로 변환**하는 단계
- 뷰프러스텀 클리핑, 투영 변환, 뷰포트 매핑 수행
- 좌표를 정규화하고 실제 화면 크기에 맞게 변환합니다. NDC 좌표계를 떠올리자.
- 프래그먼트 수를 줄이기 위한 **컬링(Culling), 클리핑, 멀티샘플링** 알고리즘을 여기서 수행



3D model



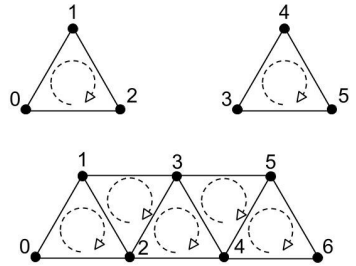


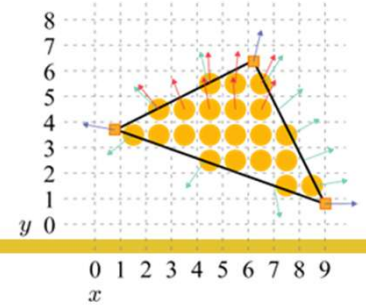


2D Space

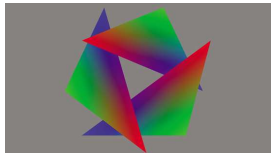
Triangle List

Triangle Strip



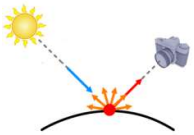


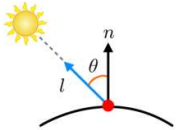
Pixel Shader Stage

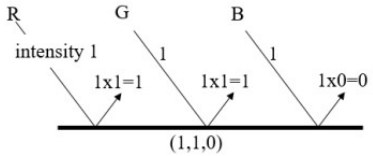


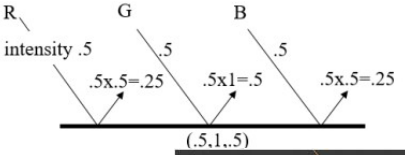
- **Rasterizer가 만든 Fragment마다 실행**되어 픽셀 색상을 결정합니다.
- 보간된 Vertex Shader 출력값, 텍스처, 상수 버퍼를 입력으로 사용합니다.
- 결과로 최종 색상(SV_Target)과 선택적 깊이(SV_Depth) 를 출력합니다.
- 픽셀 단위 연산이므로 가장 무겁고 최적화가 중요한 단계이다. 픽셀별 조명이나 후처리와 같은 기술을 넣을 때 이 단계에서 작업합니다

```
m_pDeviceContext->PSSetShader(m_pPixelShader, nullptr, 0);
```

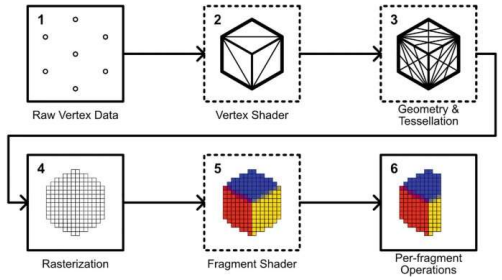




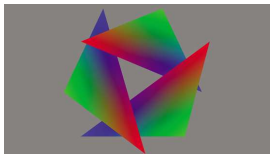




…

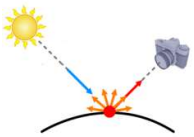


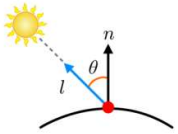
Output-Merger (OM) Stage

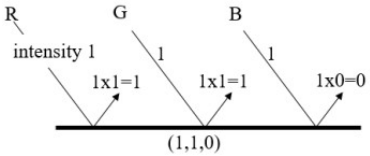


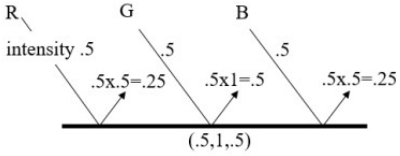
- 파이프라인 마지막 단계
- 셀 셰이더 출력값 + 기존 렌더타겟(프레임버퍼) + 깊이/스텐실 버퍼를 **조합해 최종 픽셀 색상** 결정합니다

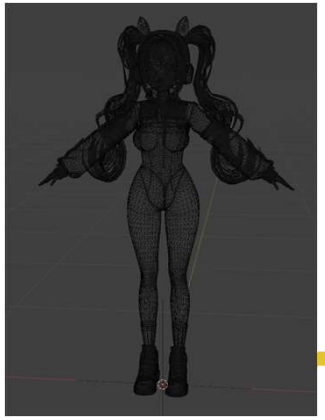
```
m_pDeviceContext->PSSetShader(m_pPixelShader, nullptr, 0);
```



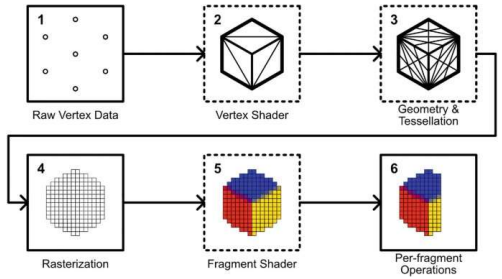




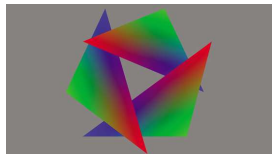




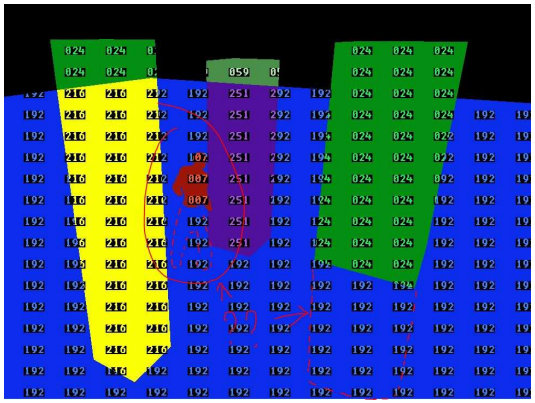
…

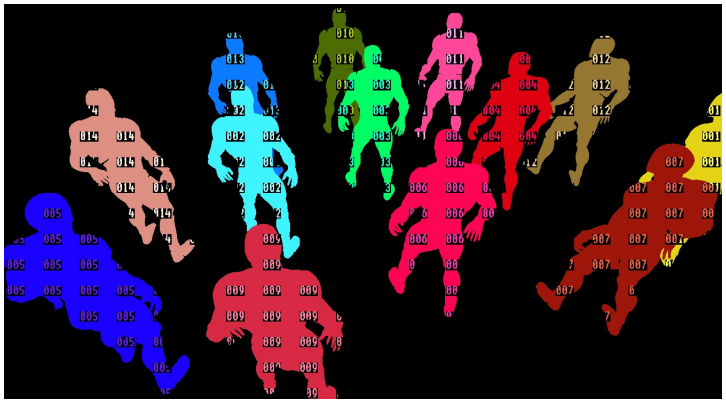


Output-Merger (OM) Stage



- 깊이/스텐실 테스트 (Depth-Stencil Test)
  - 깊이: 카메라에 더 가까운 픽셀만 남김 (Z-Buffering)
  - 스텐실: 특정 영역 픽셀을 마스킹/제어
- 블렌딩 (Blending)
  - 기존 렌더타겟 값과 새 픽셀 색상을 합성
  - 알파 블렌딩, Add/Subtract, Dual-Source blending 등 지원
- 멀티렌더타겟 (MRT)
  - 한 번의 드로우 콜에서 최대 8개의 RenderTarget에 출력 가능
- 출력 제어
  - Output-Write Mask : RGBA 개별 채널 마스크
  - Sample Mask : 멀티샘플링 시 업데이트할 샘플 지정





```
m_pDeviceContext->PSSetShader(m_pPixelShader, nullptr, 0);
```

참고

https://nikke-kr.com/mmd.html

[Vulkan Tutorial]
https://vulkan-tutorial.com/Vertex_buffers/Vertex_input_description

[Input-Assembler]
http://learn.microsoft.com/ko-kr/windows/win32/direct3d11/d3d10-graphics-programming-guide-input-assembler-stage

[vertex shader]
https://docs.tizen.org/application/native/guides/graphics/vertex-shader/
https://learn.microsoft.com/ko-kr/windows/win32/direct3d11/vertex-shader-stage

[pixel shader]
https://learn.microsoft.com/ko-kr/windows/win32/direct3d11/pixel-shader-stage
https://docs.tizen.org/application/native/guides/graphics/fragment-shader/